

EXERCISE 5

Implement Conditional Branching

ABAP Development Tools (ADT) • ABAP Cloud

CONFIDENTIALITY

The group number, service details, credentials, and other access information are not included. Class/package names are shown generically where required.

1. Exercise Objective

Extend the calculator from Exercise 4 so it can perform four arithmetic operations: addition, subtraction, multiplication, and division. Then handle invalid operators and prevent program termination when division by zero occurs.

What problem are we solving?

A program should not assume that the input is always valid. Here the operator can be +, -, *, or /. The program must choose the correct calculation, reject unsupported operators, and safely handle division by zero.

2. Starting Point

The exercise can be completed by continuing from the COMPUTE class created in Exercise 4, or by copying the template class into the development package and naming it with the course naming convention:

```
ZCL_<GROUP>_BRANCH
```

The class must implement:

```
IF_00_ADT_CLASSRUN
```

3. Task 1 — Optional: Copy the Template

- Open the ABAP Development Object search using Ctrl + Shift + A.
- Search for /LRN/CL_S4D400_BTS_COMPUTE.
- Open the template class.
- In Project Explorer, use Link with Editor if necessary.
- Right-click the template class and choose Duplicate...
- Select the development package created for the course.
- Use the BRANCH class naming convention.
- Adjust the description and assign the exercise transport request.
- Choose Finish.

4. Task 2 — Add the Operator Variable

Declare a one-character variable to store the calculation operator:

```
DATA op TYPE c LENGTH 1.
```

TYPE c is a character type. LENGTH 1 means the variable can contain one character, such as +, -, *, or /.

Set the input values and operator:

```
number1 = 123.  
number2 = 0.  
op = '/'.
```

5. Use CASE to Choose the Calculation

A CASE statement is suitable because one variable, op, determines which calculation should be executed.

```
CASE op.  
  WHEN '+'.  
    result = number1 + number2.  
  WHEN '-'.  
    result = number1 - number2.  
  WHEN '*'.  
    result = number1 * number2.  
  WHEN '/'.  
    result = number1 / number2.  
ENDCASE.
```

How CASE works

```
op = '+'  
↓  
WHEN '+'  
↓  
addition  
  
op = '-'  
↓  
WHEN '-'  
↓  
subtraction  
  
op = '*'  
↓  
WHEN '*'  
↓  
multiplication  
  
op = '/'  
↓  
WHEN '/'  
↓  
division
```

6. Update the String Template

Instead of hard-coding /, use the operator variable:

```
output = |{ number1 } { op } { number2 } = { result }|.
```

This makes the output automatically match the selected operation.

7. Invalid Operator — WHEN OTHERS

If op contains a value such as %, none of the four normal WHEN branches matches. Use WHEN OTHERS to handle all unsupported values.

```
WHEN OTHERS.  
  output = |'{ op }' is not a valid operator!|.
```

For example, if op is %, the program should produce an error instead of displaying a calculation.

```
'%' is not a valid operator!
```

8. Why Make output an Explicit Variable?

```
DATA output TYPE string.
```

The output variable must now be filled either with a calculation result or with an error message. An explicit declaration makes it available before the CASE statement.

9. Only Create the Normal Result When There Is No Error

```
IF output IS INITIAL.  
  output = |{ number1 } { op } { number2 } = { result }|.  
ENDIF.
```

If WHEN OTHERS already placed an error message in output, output is no longer initial. Therefore the normal calculation text is not created.

Simple logic

No error → output is empty → create calculation text.

Error already exists → output is not empty → keep the error text.

10. Division by Zero — The Problem

If number2 is 0 and op is /, the division statement raises the exception CX_SY_ZERODIVIDE.

```
result = number1 / number2.
```

Without exception handling, the application terminates instead of producing a controlled message.

```
CX_SY_ZERODIVIDE
```

11. Handle Division by Zero with TRY...CATCH

```
WHEN '/'.
  TRY.
    result = number1 / number2.
  CATCH cx_sy_zerodivide.
    output = |Division by zero is not defined|.
  ENDTRY.
```

TRY contains code that might raise an exception. CATCH handles a specific exception. Here the exception is caught and converted into a readable error message.

12. Complete Model Solution

```
CLASS zcl_<group>_branch DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.

  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_<group>_branch IMPLEMENTATION.

  METHOD if_oo_adt_classrun~main.

    DATA number1 TYPE i.
    DATA number2 TYPE i.
    DATA result TYPE p LENGTH 8 DECIMALS 2.
    DATA op TYPE c LENGTH 1.
    DATA output TYPE string.

    number1 = 123.
    number2 = 0.
    op = '/'.

    CASE op.

      WHEN '+'.
        result = number1 + number2.

      WHEN '-'.
        result = number1 - number2.

      WHEN '*'.
        result = number1 * number2.

      WHEN '/'.
        TRY.
          result = number1 / number2.
        CATCH cx_sy_zerodivide.
```

```

        output = |Division by zero is not defined|.
    ENDTRY.

    WHEN OTHERS.
        output = |'{ op }' is not a valid operator!|.

    ENDCASE.

    IF output IS INITIAL.
        output = |{ number1 } { op } { number2 } = { result }|.
    ENDIF.

    out->write( output ).

    ENDMETHOD.

ENDCLASS.

```

13. Test Cases

number1	number2	op	Expected result
10	5	+	10 + 5 = 15
10	5	-	10 - 5 = 5
10	5	*	10 * 5 = 50
10	5	/	10 / 5 = 2.00
10	5	%	'%' is not a valid operator!
10	0	/	Division by zero is not defined

14. Screenshot Evidence

Screenshot note

No Exercise 5 screenshot file was available in the current document-generation workspace. No unrelated screenshot was inserted. When the Exercise 5 screenshots are available, they can be placed beside the corresponding steps.

15. Technical Vocabulary

Term	Simple meaning
Conditional branching	Choosing different code paths depending on a condition/value.
CASE	ABAP control structure that chooses a branch based on an expression.
WHEN	Defines one possible value/branch inside CASE.
WHEN OTHERS	Handles all CASE values not matched by earlier WHEN branches.
Operator	A symbol that tells the program which arithmetic operation to perform.
Exception	An abnormal situation raised while a program is running.

CX_SY_ZERODIVIDE	ABAP exception raised for division by zero.
TRY	Marks code that may raise an exception.
CATCH	Handles a specified exception.
ENDTRY	Ends the TRY...CATCH block.
TYPE c LENGTH 1	One-character text variable suitable for +, -, *, or /.
IF output IS INITIAL	Checks whether output currently contains no value.

16. Why This Exercise Matters in a Company

- Business applications receive different inputs and cannot assume every value is valid.
- CASE/IF logic is used for business rules and different processing paths.
- Invalid input must be handled safely instead of producing misleading output.
- Exceptions must be handled so a normal user action does not unnecessarily terminate the application.
- Division-by-zero handling is a simple example of defensive programming.
- Readable error messages make troubleshooting and support easier.

Real-company example

Imagine an order-processing program that chooses pricing logic based on an order type. If the type is unknown, the program should report a controlled error. If a calculation can fail, the exception should be handled rather than crashing the process.

17. Interview Questions and Answers

1. Why use CASE here?

The calculation depends on one operator variable with several known possible values, so CASE clearly expresses the alternatives.

2. What does WHEN OTHERS do?

It handles all values that were not matched by the earlier WHEN branches.

3. What happens if op is %?

No normal arithmetic branch matches, so WHEN OTHERS sets an invalid-operator error message.

4. What is an exception?

A runtime condition that interrupts normal program flow because an operation cannot be completed normally.

5. What exception occurs when dividing by zero?

CX_SY_ZERODIVIDE.

6. Why use TRY...CATCH?

To catch and handle a runtime exception instead of allowing the application to terminate unexpectedly.

7. What does CATCH cx_sy_zerodivide do?

It catches the division-by-zero exception and allows the program to set a controlled error message.

8. Why check IF output IS INITIAL?

To prevent the normal result from overwriting an error message already stored in output.

9. What is defensive programming?

Writing code that anticipates invalid inputs and failure situations and handles them safely.

10. Why is exception handling important in SAP?

Business applications must remain reliable when unexpected data or runtime conditions occur.

18. What Was Performed — Final Summary

- Added an operator variable.
- Used CASE to support +, -, *, and /.
- Updated the output string to use the operator dynamically.
- Added WHEN OTHERS for invalid operators.
- Made output an explicit string variable.
- Used IF output IS INITIAL to preserve error messages.
- Handled CX_SY_ZERODIVIDE with TRY...CATCH.
- Tested valid operations, invalid operators, and division by zero.
- Activated and executed the class in ADT.

Core lesson

A good ABAP program does not only handle the successful path. It must also control invalid input and runtime errors.